

CS-338 Intro to the Theory of Computation

- Lecture #8

➤ Church-Turing Thesis

- Prof Pietro S. Oliveto
Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)
- `oliveto@ust.hk`
<https://faculty.ust.hk/~oliveto>
- Office: Room 113

Overview

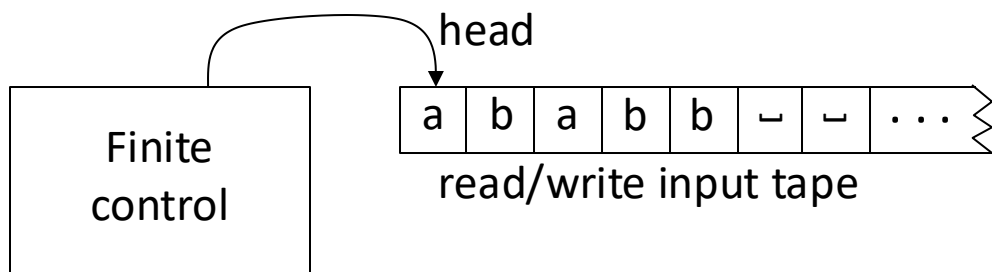
Last time:

- Proving languages not Context Free
- Turing machines
- Recognizers and deciders
- T-recognizable and T-decidable languages

Today: (Sipser §3.2 – §3.3)

- Variants of the Turing machine model
 - a. Multi-tape TMs
 - b. Nondeterministic TMs
 - c. Enumerators
- Church-Turing Thesis
- Notation for encodings and TMs

Turing machine model – review



On input w a TM M may halt (enter q_{acc} or q_{rej}) or loop (run forever).

So M has 3 possible outcomes for each input w :

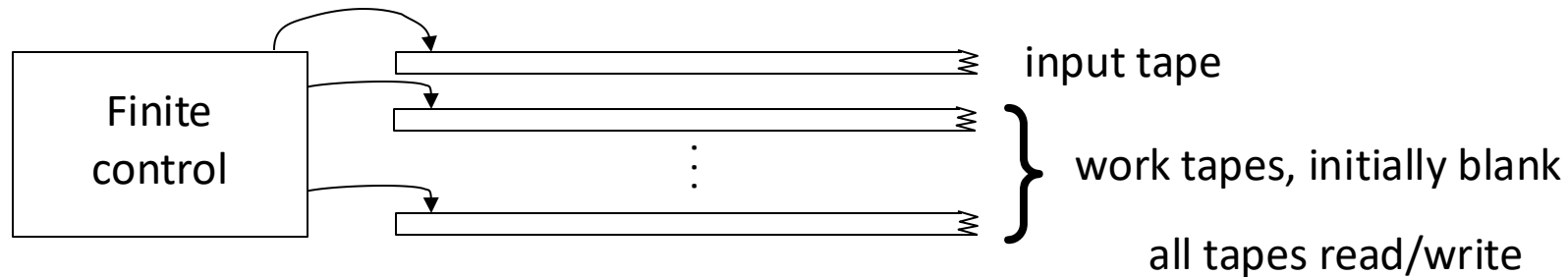
1. Accept w (enter q_{acc})
2. Reject w by halting (enter q_{rej})
3. (Reject) w by looping (running forever)

A is T-recognizable if $A = L(M)$ for some TM M .

A is T-decidable if $A = L(M)$ for some TM decider M .

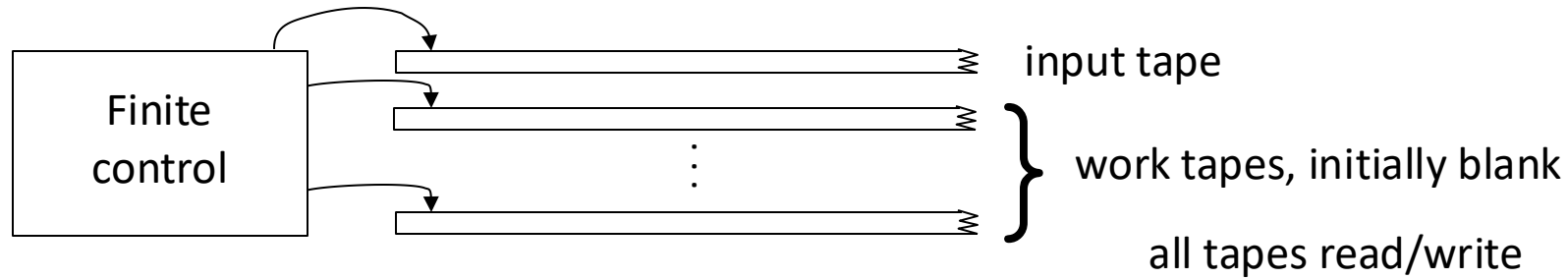
halts on all inputs ↗

Multi-tape Turing machines



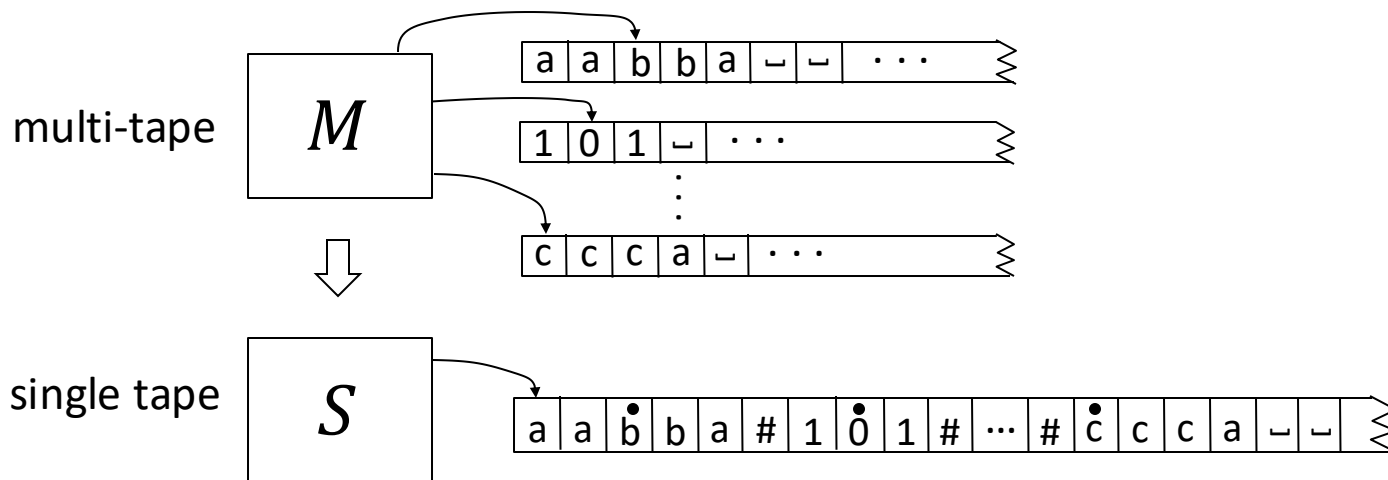
- Like an ordinary Turing Machine but with multiple tapes
- Each tape has its own head for reading and writing
- The transition function allows for reading, writing and moving the heads of all tapes simultaneously:
 - $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$, (Left, Right, Stay put)
where k is the number of tapes.
 - Eg., $\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, L, R, \dots, S)$
- Is it more powerful than a single tape TM?
- Is a single tape TM with the S move more powerful than one without?
- Is a PDA with more stacks more powerful than a PDA with one stack?
 - Yes with two stacks you can recognise the language $L = \{0^k, 1^k, 2^k \mid k \geq 0\}$ (exercise)

Multi-tape Single-tape Turing machine equivalence



Theorem: A is T-recognizable iff some multi-tape TM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert multi-tape to single tape:



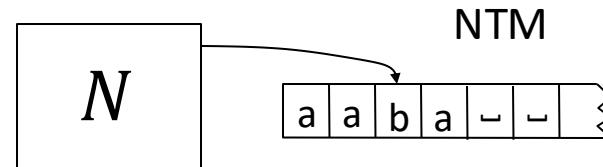
S simulates M by storing the contents of multiple tapes on a single tape in “blocks”. Record head positions with dotted symbols.

Some details of S :

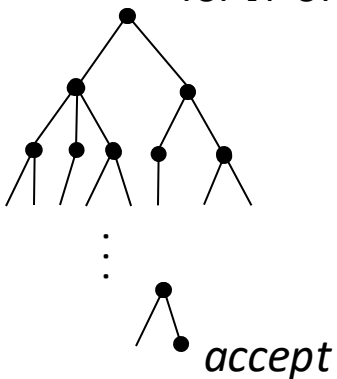
- 1) To simulate each of M 's steps
 - a. Scan entire tape to find dotted symbols.
 - b. Scan again to update according to M 's δ .
 - c. Shift to add room as needed.
- 2) Accept/reject if M does.

Nondeterministic Turing machines

A Nondeterministic TM (NTM) is similar to a Deterministic TM except for its transition function $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$.



Nondeterministic computation tree
for N on input w .

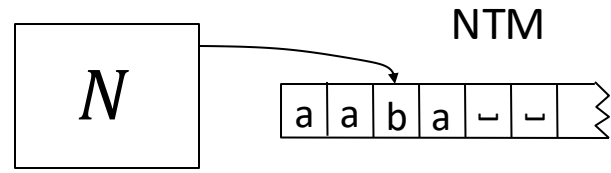


- At any point in a computation the machine may proceed according to different choices
- If some branch of the computation leads to an accept state, then the machine accepts its input
- Which is more powerful NTM or DTM?
- Which is more powerful DFA or NFA?
- Which is more powerful (N) PDA or DPDA?

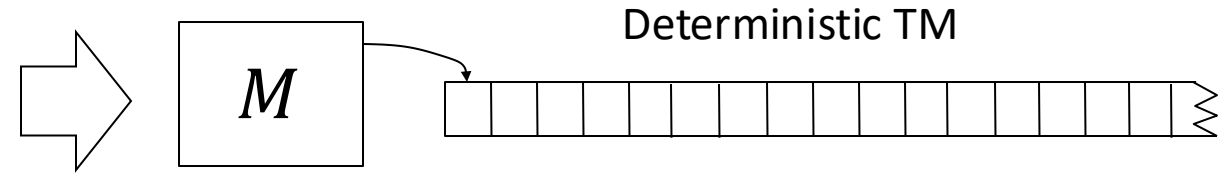
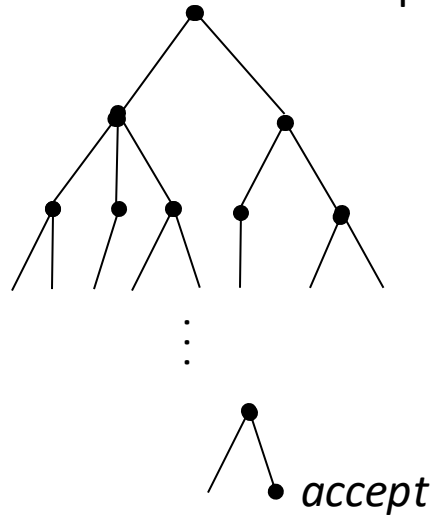
Nondeterministic - deterministic TM equivalence (1)

Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.



Nondeterministic computation tree
for N on input w .



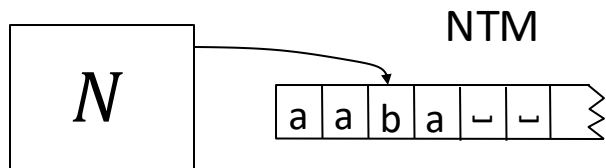
- M has to explore all the branches of the tree of N 's computation.
- If one branch accepts, then the M accepts.
- How do we proceed?
 - Depth First search?
 - No, we could go forever in an infinite branch (loop)
 - $\Rightarrow$ Breadth first search?
 - Guarantees we explore every node on the "frontier" before going deeper i.e., we don't miss any potentially accepting branch

Nondeterministic - deterministic TM equivalence (2)

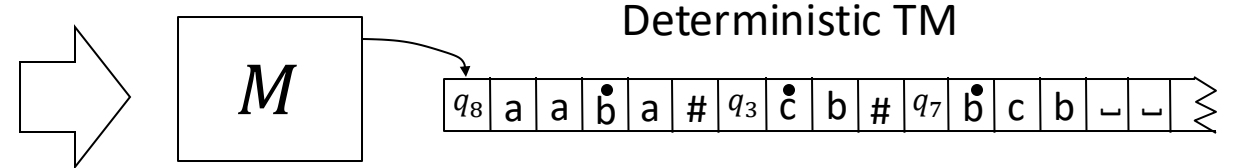
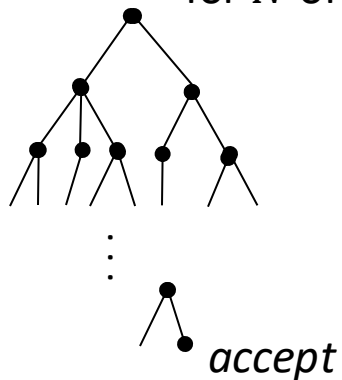
A Nondeterministic TM (NTM) is similar to a Deterministic TM except for its transition function $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$.

Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.



Nondeterministic computation tree for N on input w .



M simulates N by storing each thread's tape in a separate "block" on its tape.

Also need to store the head location, and the state for each thread, in the block.

If a thread forks, then M copies the block.

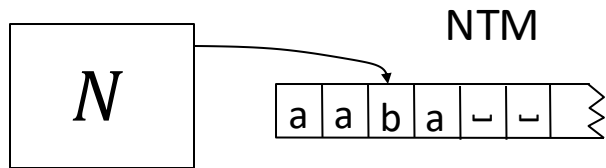
If a thread accepts then M accepts.

NTM – DTM equivalence: traditional proof

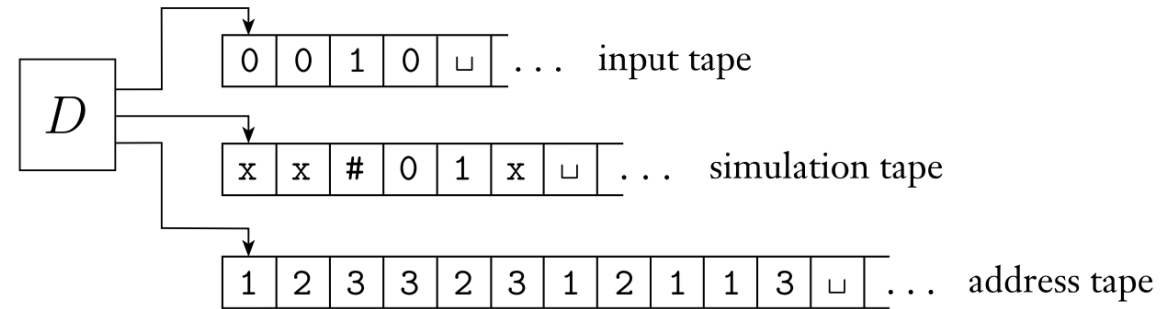
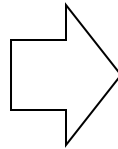
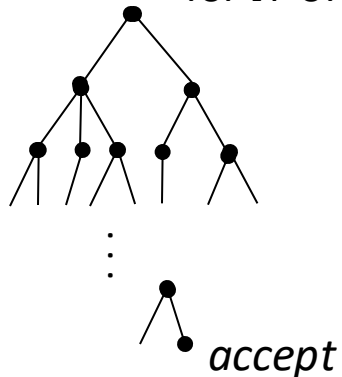
A Nondeterministic TM (NTM) is similar to a Deterministic TM except for its transition function $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$.

Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.



Nondeterministic computation tree
for N on input w .



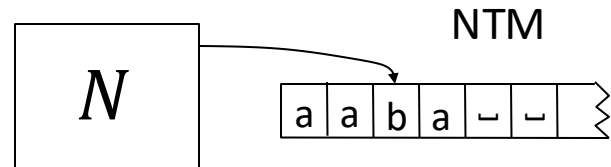
Address tape contains alphabet $\Gamma_b = \{1, 2, \dots, b\}$ (b = largest set of choices)

1. Tape contains input; tape 2 is empty; tape 3 contains ϵ .
2. Copy tape 1 to tape 2;
3. Simulate N on tape 2 using the string on tape 3 to make choices;
 - if accept configuration $\Rightarrow$ **accept**
 - if a choice is invalid abort $\Rightarrow$ step 4;
 - If reject configuration $\Rightarrow$ step 4;
4. increase lexicographically the string on tape 3 $\Rightarrow$ step 2

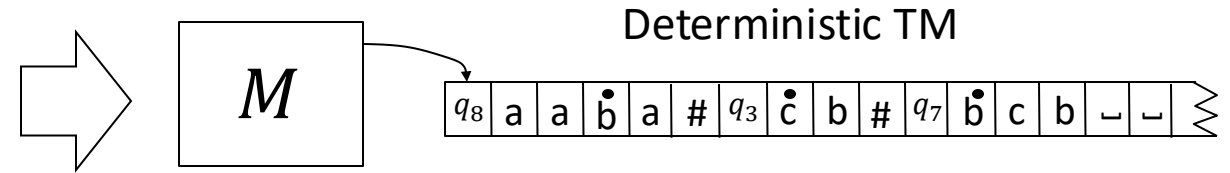
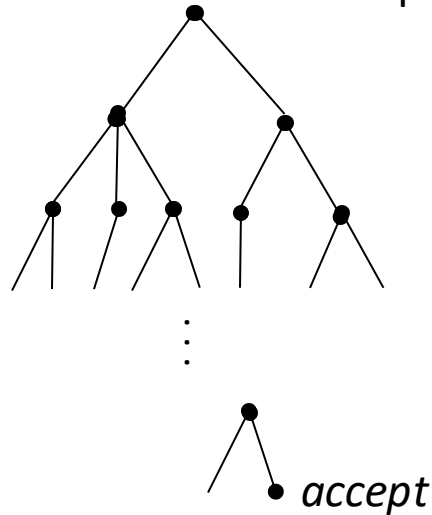
Nondeterministic TM Deciders

Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.



Nondeterministic computation tree
for N on input w .

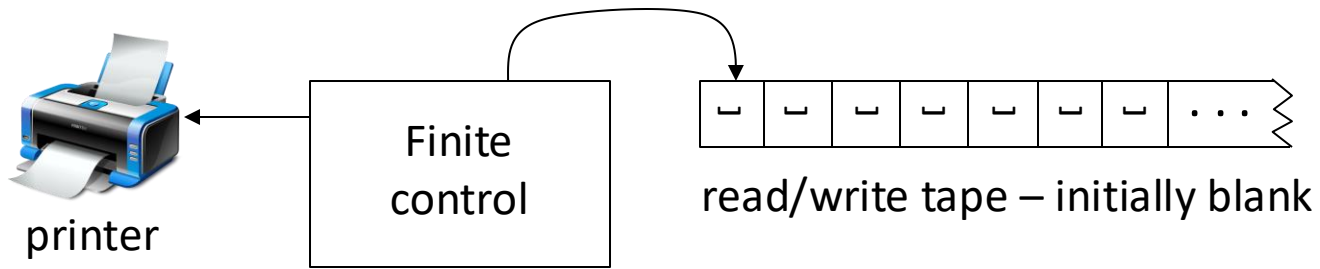


- If all computation branches of the NTM **halt** on either *accept* or *reject* configurations for all inputs, then the NTM is a **decider**

Corollary

A language is decidable if and only if some nondeterministic Turing machine decides it.

Turing Enumerators



Defn: A Turing Enumerator is a deterministic TM with a printer.

It starts on a blank tape and it can print strings $w_1, w_2, w_3, \dots$ possibly going forever.

Its language is the set of all strings it prints. It is a **generator**, not a recognizer.

For enumerator E we say $L(E) = \{w \mid E \text{ prints } w\}$.

Theorem: A is T-recognizable iff $A = L(E)$ for some T-enumerator E .

Proof: ($\leftarrow$) Convert E to equivalent TM M .

M = for input w :

Simulate E (on blank input).

Whenever E prints x , test $x = w$.

Accept if = and continue otherwise.

Proof: ($\rightarrow$) Convert TM M to equivalent enumerator E .

E = Simulate M on each w_i in $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, \dots\}$

If M accepts w_i then print w_i .

Continue with next w_i .

Problem: What if M on w_i loops?

Fix: Simulate M on $w_1, w_2, \dots, w_i$ for i steps, for $i = 1, 2, \dots$

Print those w_i which are accepted.

Summary: Which is more powerful?

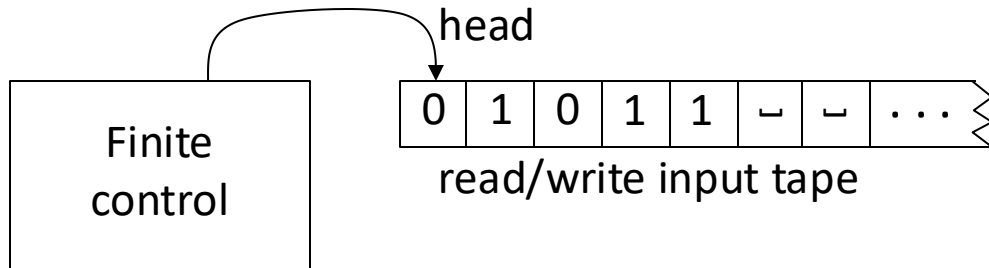
- Turing Machines, or:
- DFA, NFA, regular expressions?
- PDA, CFGs?
- Multi-tape TMs, NTMs, Enumerators?
- Random Access Machines?
- Your laptop?
- High-performance computer?

What do computers actually do?

- They use algorithms of some kind to solve problems
- Algorithms are essentially a series of math operations of some kind
- Eg., adding 1 to a counter, storing something somewhere in some memory, multiplying two numbers etc.

Increment Operations

- Can a TM take an input number x and add 1?

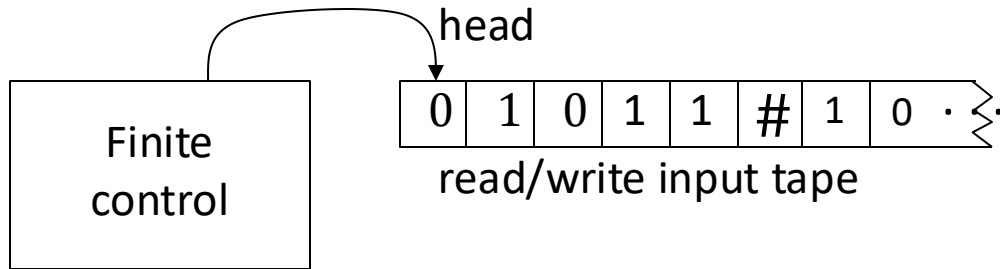

$$\begin{array}{r} 01011 \quad + \\ \quad 1 \\ \hline 01\mathbf{100} \end{array}$$

TM Algorithm:

- *“Start from the end of the string going left, while we see a 1 write a 0. Once we see a 0 write a 1. If no first 0, shift everything to the write and add a 1 at the beginning”*
- **Decrements** are analogous

Additions

- Can a TM add x and y ?



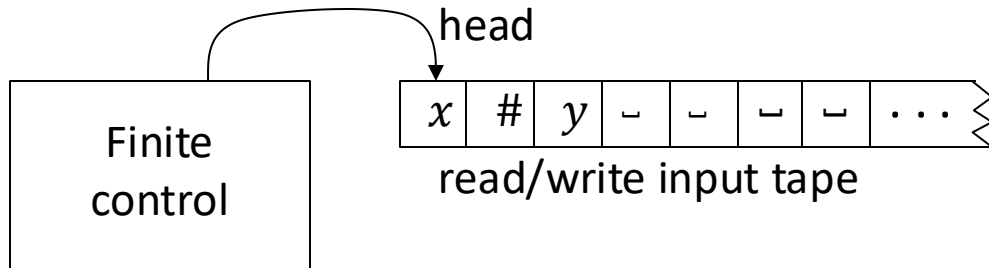
x		y
01011	#	10
01010	#	11 (subtract one from x and add it to y)
01001	#	100 (repeat)
01000	#	101
00111	#	110
00110	#	111
00101	#	1000
...		
00000	#	01101

TM Algorithm:

- “Decrement one from x and add one to y until x is zero. Return y ”*

Multiplications

- Can a TM multiply x and y ?



$$x * y = y + y + y + \dots + y \text{ (} x \text{ times)}$$
$$(4 * 3 = 3 + 3 + 3 + 3)$$

Repeated additions:

TM Algorithm:

- “Add y to y , x times”
-
- Can a TM do x^y ?
 - Repeated multiplications...
 - You can simulate the X86 instruction set
 - If you do infinite numbers, irrational numbers etc. computers approximate still using binary numbers for representation still involving additions, multiplications, subtractions and divisions.

Summary: Which is more powerful?

- Turing Machines, or:
- DFA, NFA, regular expressions? **TM**
- PDA, CFGs? **TM**
- Multi-tape TMs, NTMs, Enumerators? **Equivalent**
- Random Access Machines? **Equivalent**
- Your laptop? **Equivalent**
- High-performance computer? **Equivalent**

Church-Turing Thesis (Algorithm Definition)

- We started the course with the following question: “*What is a computer?*”
- What is an algorithm? It’s a Turing Machine



Alonzo Church
1903–1995

Algorithm

Intuitive

Instead of Turing machines,
can use any other “reasonable” model
of unrestricted computation:
 λ -calculus, random access machine,
your favorite programming language, ...

=

Turing
machine

Formal



Alan Turing
1912–1954

Why no Proof?

Because an algorithm is an intuitive concept:

- X86 instruction set (data movement, math operations, control flow)?
- All possible instructions?
- All instructions done in a finite number of steps

⇒ If a problem is solvable with a TM then I can
solve it with my favourite computer

⇐ If a problem can’t be solved by a TM, then
no computer can solve it

Church-Turing Thesis (2)

- Church's Thesis (1936): *"A function of positive integers is effectively calculable* only if λ -definable"* – (by using lambda calculus)
- Turing's Thesis (1936): *"Every computation that can be carried out by mechanical means can be programmed on a Turing Machine"*
- Turing (1936) also proved that "all lambda-definable sequences[...] are computable" – The two definitions are equivalent
- To prove false:
 - Find some procedure for which we can write a computer program for which we can show no TM exists
- But:
 1. Everything that can be computed on any existing digital computer can also be computed on a TM
 2. No one has found a problem solvable by an "algorithm" for which a TM program cannot be written
 3. Many reasonable** alternative models have proposed for computation but none of them is more powerful than a TM

* Computable in finite time

** Unrestricted access to unlimited memory & ability to perform only a finite amount of work in one step

Church-Turing Thesis (3)

- Newton's laws of motion, Darwin's evolutionary model etc. are not proven: they are plausible models that explain the physical world
- For similar reasons, the Church-Turing thesis is the basic law of computer science
- **Definition**
An **algorithm** for a function $f: D \rightarrow R$ is a Turing machine M that given $d \in D$ as input on its tape eventually halts with the correct answer $f(d)$ on its tape, for all $d \in D$.
- This allows us to prove rigorously claims such as:
 - *"there exists an algorithm..."*
 - *"no algorithm exists that..."*

Hilbert's 10th Problem

In 1900 David Hilbert posed 23 problems

- #1) Problem of the continuum (Does set A exist where $|\mathbb{N}| < |A| < |\mathbb{R}|$?).
- #2) Prove that the axioms of mathematics are consistent.
- #10) Give an algorithm* for solving *Diophantine equations*.

Diophantine equations:

Equations of polynomials where solutions must be integers.

Example: $3x^2 - 2xy - y^2z = 7$ solution: $x = 1, y = 2, z = -2$

Let $D = \{p \mid \text{polynomial } p(x_1, x_2, \dots, x_k) = 0 \text{ has a } \underline{\text{solution in integers}}\}$

Hilbert's 10th problem: Give an algorithm to decide D .

Only after the Church-Turing thesis was it possible to prove that an algorithm does not exist to solve Diophantine equations.

Matiyasevich proved in 1970: D is not decidable.

Note: D is T-recognizable.



David Hilbert
1862—1943

* “a process according to which it can be determined by a finite number of operations”

Hilbert's 10th Problem (2)

Diophantine equations:

Equations of polynomials where solutions must be integers.

Example: $3x^2 - 2xy - y^2z = 7$ solution: $x = 1, y = 2, z = -2$

Let $D = \{p \mid \text{polynomial } p(x_1, x_2, \dots, x_k) = 0 \text{ has a } \underline{\text{solution in integers}}\}$

Hilbert's 10th problem: Give an algorithm to decide D .

Theorem: D is not decidable.

Proof Sketch. Let

$D_1 = \{p \mid p \text{ is a polynomial over } x \text{ with an integral root}\}.$

Here is a TM M_1 that recognizes D_1 :

$M_1 =$ “On input $\langle p \rangle$: where p is a polynomial over the variable x .

1. Evaluate p with x set successively to the values $0, 1, -1, 2, -2, 3, -3, \dots$. If at any point the polynomial evaluates to 0, *accept*.”

M_1 can be converted into a **decider** by calculating bounds within the roots of a polynomial of a single variable must lie i.e., $\pm k \frac{c_{max}}{c_1}$, (k is number of terms)

Matiyasevich proved that calculating such bounds for multivariable polynomials is impossible!



David Hilbert
1862—1943

Notation for encodings and TMs

- The input of a TM is always a string
- If we want to provide some different object in input we must first convert it to a string
- Strings can easily represent polynomials, graphs, grammars, automata etc. and any combinations of those objects

Notation for encoding objects into strings

- If O is some object (e.g., polynomial, automaton, graph, etc.), we write $\langle O \rangle$ to be an encoding of that object into a string.
- If $O_1, O_2, \dots, O_k$ is a list of objects then we write $\langle O_1, O_2, \dots, O_k \rangle$ to be an encoding of them together into a single string.

Notation for writing Turing machines

We will use high-level English descriptions of algorithms when we describe TMs, knowing that we could (in principle) convert those descriptions into states, transition function, etc. Our notation for writing a TM M is

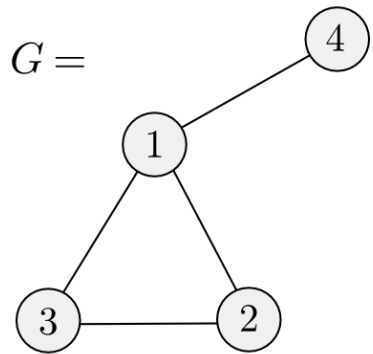
$M =$ “On input w
[English description of the algorithm]”

Encoding Example

Let A be the language consisting of all strings representing undirected graphs that are connected. Recall that a graph is **connected** if every node can be reached from every other node by traveling along the edges of the graph. We write

$$A = \{\langle G \rangle \mid G \text{ is a connected undirected graph}\}.$$

Encoding:



$\langle G \rangle =$

$(1, 2, 3, 4) ((1, 2), (2, 3), (3, 1), (1, 4))$

Turing Machine operation:

- First M checks that it has a graph in input (a proper list of nodes and a proper list of edges, followed by blanks)
- Then it executes the algorithm to check whether the input is a connected graph

$M =$ “On input $\langle G \rangle$, the encoding of a graph G :

1. Select the first node of G and mark it.
2. Repeat the following stage until no new nodes are marked:
 3. For each node in G , mark it if it is attached by an edge to a node that is already marked.
4. Scan all the nodes of G to determine whether they all are marked. If they are, *accept*; otherwise, *reject*.”

TM – example revisited

Turing Machine algorithms can be described:

- Formal description (7-tuple)
- Implementation description: we describe how the TM operates in English (英文) prose (last week's exercise)
- High-level description: pseudo-code

TM M recognizing $B = \{a^k b^k c^k \mid k \geq 0\}$

M = “On input w

1. Check if $w \in a^* b^* c^*$, *reject* if not.
2. Count the number of a 's, b 's, and c 's in w .
3. *Accept* if all counts are equal; *reject* if not.”

High-level description is ok.

You do not need to manage tapes, states, etc...

Quick review of today

1. We showed that various TM variants (multi-tape, nondeterministic, enumerator) are all equivalent to the single-tape model.
2. Concluded that all “reasonable” models with unrestricted memory access are equivalent.
3. Discussed the Church-Turing Thesis:
Turing machines = algorithms
4. Notation for encoding objects and describing TMs.